

Tema 2: Temas complementarios**Título:** Genericidad y Genericidad Restringida**Anexo I:** Guía de ejercicios**Ejercicio 1**

Cree la siguiente clase en C#, Java o C++ de acuerdo al lenguaje estudiado.

```
public class Pair<T,U>
{
    public T Fst;
    public U Snd;

    public Pair(T fst, U snd) {
        this.Fst = fst;
        this.Snd = snd;
    }
    public override String ToString()
    {
        return "(" + Fst + ", " + Snd + ")";
    }
}
```

- En el programa principal instancie un objeto de la clase **Pair** con los parametros **<String, int>**, asígnele el valor que desee.
- En el programa principal cree una lista de instancias de la clase **Pair** utilizando la clase *Lista* vista en clases.
- Añada elementos a la lista y muéstrellos usando las funciones correspondientes.
- En el programa principal declare una variable llamada **punto** de tipo **Pair<Pair<int,int>,String>** y asígnele el valor que desee. ¿Cuál será el tipo de *punto.Fst.Snd*?

Ejercicio 2.

Declare una clase llamada *PairNumber* que herede de la clase *Pair<int, int>*, a esta clase agréguele un método llamado *comparar* que permita, dado un objeto de tipo *Pair<int, int>* compararlo con el objeto actual.

Dos objetos de tipo *Pair<int, int>* a y b, son comparables de la siguiente forma:

a es mayor que b si $a.Fst > b.Fst$

a es menor que b si $a.Fst < b.Fst$

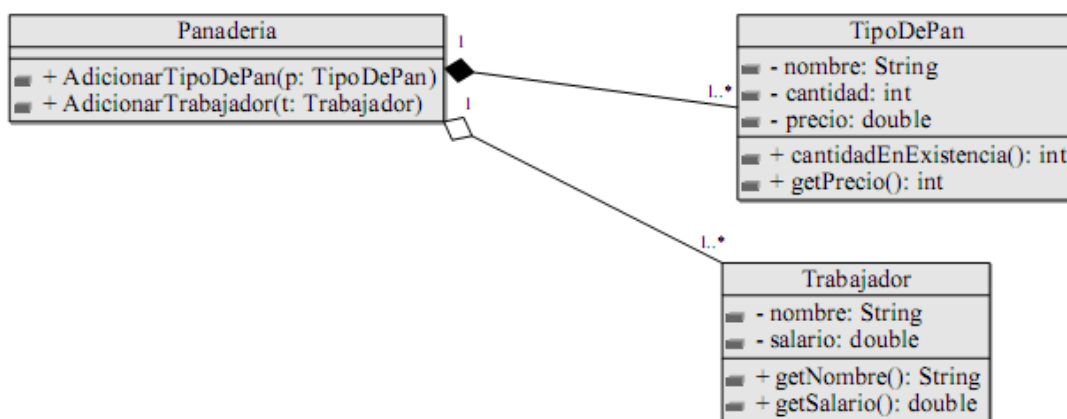
y son iguales en caso de que $a.Fst = b.Fst$

- Cree una clase llamada **ListaPairNumber** que herede de la clase **Lista** vista en conferencia pero con el tipo de parámetro **PairNumber**
- Implemente un método ordenar en la clase del inciso anterior que permita ordenar de mayor a menor los elementos de la lista. Para ello debe utilizar el método comparar de la clase **PairNumber**.

Ejercicio 3

En una panadería se necesita un sistema para controlar a sus trabajadores y la producción de pan. En la misma se tiene un registro de cada trabajador (nombre y salario) y de cada tipo de pan que produce (precio y cantidad en existencia).

El diagrama siguiente representa una modelación del problema:



- Declare la clase Panadería utilizando la clase genérica `Lista<T>` e incluya en la misma métodos para cumplir con las siguientes funcionalidades:
- Determine el nombre del trabajador de mayor salario.
- Diga cuantos panes de cada tipo se pueden comprar con el salario del trabajador que más cobre.

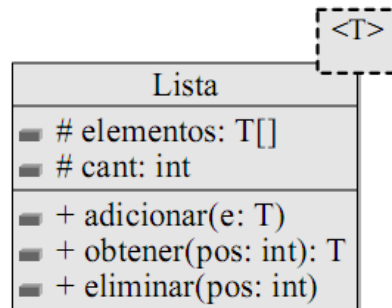
Ejercicio 4

Declare una clase genérica llamada **ConjuntoGeneral** que herede de la clase **Lista** vista en clases con la condición de que puedan existir elementos repetidos en la lista.

- Agregue un método a la clase que sume dos elementos cualesquiera del conjunto dadas sus posiciones (si son números es la suma y si son cadenas la suma es la unión o concatenación de las dos cadenas).
- Agregue un método que dado un elemento lo elimine del conjunto si se encuentra repetido varias veces.

Ejercicio 5

Construya una clase que herede de Lista<T> (vista en clases) y que permita manejar objetos de cualquier tipo:

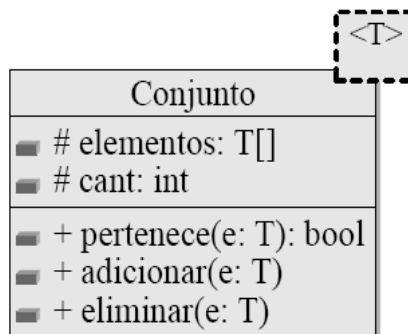


Impleméntele métodos para:

- a) Intercambiar los elementos de las posiciones impares con el que esté a continuación.
- b) Dada una cantidad, devolver un elemento que esté más de esa cantidad de veces en la lista.

Ejercicio 6

A partir de la implementación de la clase Conjunto representada en el siguiente diagrama:



Implemente una clase (también genérica) que herede de Conjunto< T > y que permita:

- a) Mantener un registro de los elementos que se eliminen del conjunto (almacenar los elementos que se vayan eliminando).
- b) Contar las veces que se pregunte por la pertenencia al conjunto de un objeto que se asume está en un atributo de nombre **obj**¹.

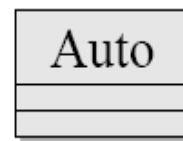
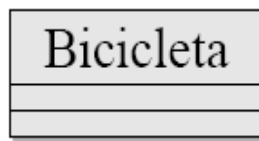
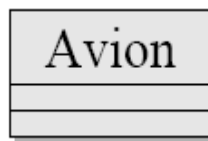
Ejercicio 7

¹ Deben incluir en la clase un atributo obj de tipo T (T obj;)

A partir de sus conocimientos de Programación Orientada a Objetos y de las capacidades de implementación de clases genéricas del lenguaje que Ud utiliza implemente una clase para manipular tres objetos de cualquier tipo, la misma debe poseer métodos de acceso y un constructor.

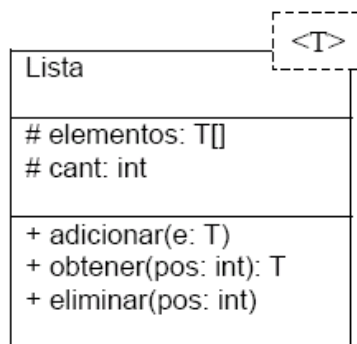
a) Sean Avión, Bicicleta y Auto clases ya implementadas (representadas en la figura siguiente), construya una instancia de la clase desarrollada en el inciso anterior que tenga un objeto avión y dos objetos auto.

b) Implemente un método que intercambie los objetos auto entre sí.



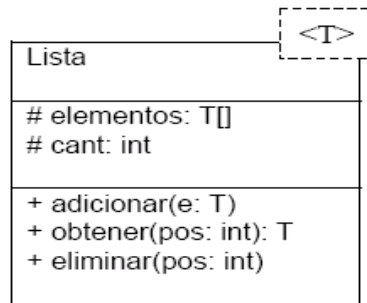
Ejercicio 8

Construya una clase que herede de Lista<T> (vista en clases) y que permita manejar objetos de cualquier tipo, con la particularidad de que a la hora de eliminar un elemento, elimine siempre el que se encuentra en la posición impar anterior a la que el usuario desea.



Ejercicio 9

Construya una clase que herede de Lista<T> (vista en clases) y que permita manejar objetos de cualquier tipo, con la particularidad de que a la hora de adicionar un elemento se adicione en la posición del medio en caso de tener una cantidad impar de elementos y al inicio en caso contrario.



Ejercicio 10

Declare una clase genérica **Grafo<T>** que permita modelar un grafo con las características vistas en la conferencia de teoría de grafos en la asignatura Matemática Discreta. El grafo debe tener una matriz de adyacencia y una lista genérica de vértices, en el que los vértices serían elementos de cualquier tipo de datos T. Utilice la clase Lista vista en clases.

Agregue los siguientes métodos a la clase Grafo<T>

- Crear el grafo con la cantidad de vértices especificado por el usuario.
- Obtener la matriz de adyacencia del grafo.
- Obtener el listado de vértices del grafo
- Insertar un nuevo vértice en el grafo(tener en cuenta que se debe redimensionar la matriz de adyacencia además de insertar un nuevo vértice en la lista de vértices)
- Dado dos vértices, determinar si son adyacentes.
- Dado un vértice hallar el grado del mismo (Es el número de aristas incidentes en él)
- Dado un camino, determine si es un camino simple.(Un camino en el que no se repiten los vértices)
- Dado un camino (lista de vértices), verificar si el camino es válido a partir de la definición 2.9 de la conferencia de grafos.

Cree una aplicación visual con un menú que permita realizar las opciones anteriores.